Week 3 - Physical Data Modelling

2.1 - Physical Data Modelling

Data Modelling Process

Real World → ER Model (Diagrams) → Relational Model (Tables) → DBMS

Conceptual — Physical

Physical data model

- A model used by an actual database system
  - ex: IMS: Hierarchical Model
  - MySQL: Relational Model
  - Objectivity/DB: Object-oriented
  - PostgreSQL: Object-relational model
  - sedna: XML Model
  - MongoDB: Document (JSON) model
  - Redis: Key-value pair
  - Neo4j: Graph model

Conceptual vs Physical

ER Model vs Relational model

- Both are used to model data.
- ER model
  - Has more concepts: entities and relationships
  - closer to real world and our thinking
  - no computation
- Relational Model
  - Just one concept: relation
  - Good for efficient storage and manipulation.
  - Equipped with algebra and operations to define computation

2.2 Physical Data Models

History

Pre-relational

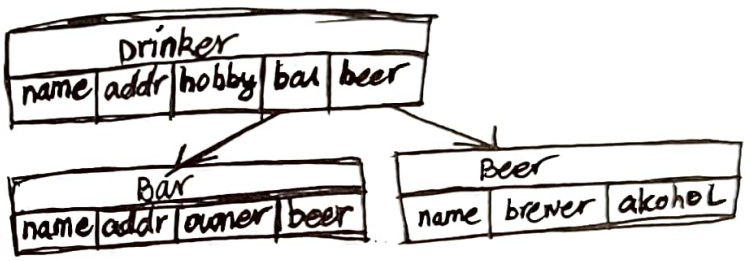
- 1960s Hierarchical Model
- 1960s Network Model
- 1970s Relational Model

Post-relational

- 1980s Object-oriented
- 1980s Object-Relational
- 1990s Document model
- 1990s Key-Value Model
- 2000s Graph Model

Hierarchical Model

- Records connected in hierarchies.
- Data accessed by traversing the hierarchies.



Network Model

- Inspired by hierarchical model, addressing many restrictions.
- Extending tree-like hierarchies to general networks.
- Data accessed by navigating networks, from any node.

Relational Model

- Motivation: Network model encodes access pathes: fragile and inefficient.
- Records stored in relations without fixed inter-connections.
- Data accessed by computing over relations.
- Most popular: MySQL, PostgreSQL

σ (Drinkers)

Drinkers $\bowtie$ Bars

Post-relational Models: Motivations

- Meeting Programming Paradigms
  - ↳ Driven by the "impedence-mismatch" with object-oriented programming.
- Dealing with data in various new settings.
  - ↳ Document, key-value, graph.
  - ↳ Driven by applications beyond enterprise data management

2.3 Relational Model

Example of a Relation

Attributes, Fields, Columns

students

id	name	major	birthday
1	Bugs Bunny	CS	2004-11-06
2	Donald Duck	Bio	1997-02-01
3	Peter Pan	Econ	1998-10-01
4	Mickey Mouse	CS	1995-04-01

Tuples, records, rows

Each attribute has a data type.

Relation is a set of tuples of the same structure.

The set of attributes is sufficient to distinguish one tuple from another.

Domains

- Each attribute has a datatype, called its domain.
- Must be atomic type -- simple values, no further structure.
- Common types: integer, string, real.
- Ex:
  - SQLite
    - text, integer, real, blob
  - MySQL
    - char, varchar, int, float, date, time, year, ...

Schemas of Relations and Databases

- Schema of a Relation
  - Relation name, attribute names, and their domains.
    - Students(id: string, name: string, major: enum('cs', 'ece', 'ss', 'music'), birthdate: date)
  - May omit domains in notation if they are clear.
    - Students(id, name, major, birthdate)
- Schema of a DATABASE
  - A set of relation schemas
    - Students(id, name, major, birthdate)
    - Professors(id, name, ~~major~~ dept, ~~birthdate~~ course)
    - Courses(number, title, credit)
    - .....

Constraints: Part of a Schema

- Key Constraint
  - Unique: Attributes must be unique among tuples in the table
  - Primary Key: Attributes are the primary key of the table.
- Null Constraint
  - Not NULL: Attributes cannot be missing/unspecified.
- Default Constraint
  - The default value will be added to records if no other value is specified.

Specifying and showing schemas

1)
```
CREATE TABLE students (
        id char(10),
        name varchar(100)
        ⋮
);
```

2) `DESCRIBE students;`

Instances of a Schema

- Relational schema $R(A_1, \ldots, A_k)$
- Instance: Set of tuples for the relation.
- Database schema $R_1(\ldots), \ldots, R_n(\ldots)$
- Instance = instances of $R_1, R_2, \ldots, R_n$

Updates: Changing Instances

- The database maintains a current state.
- updates to instance: Frequent.
  - Add a tuple.
  - Delete a tuple.
  - Modify a tuple.

```
INSERT INTO Students VALUES ('1', 'Bugs', 'CS', '2004-11-2
```

```
SELECT * FROM students;
```

Updates: Changing Schemas

- Updates to schema: Infrequent
  - Add/delete an attribute
  - Change domains of an attribute
  - Add/delete a table.
- Think of it as columns vs rows of a table.
  - Rows change much more frequently than columns.

```
ALTER TABLE students MODIFY id int;
ALTER TABLE students ADD hobby char(30);
```

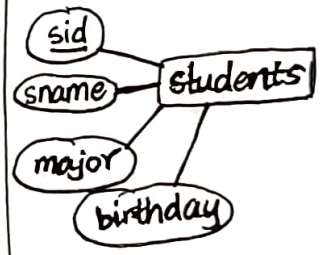
2.4 From ER to Relational Model

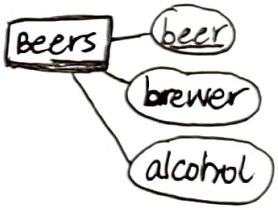
ER diagram → Relational Schema
Combine relations to simplify schema.

Change some attributes to avoid clashes.

Rules for translation

Rule 1: Entity Set → Relation



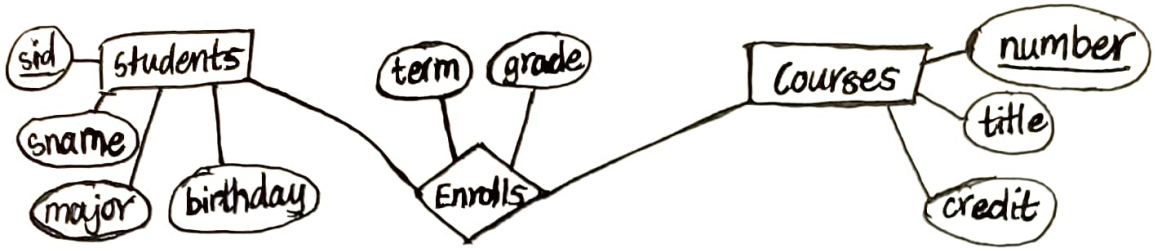


⇓

Students(sid, sname, major, birthday) Beers(beer, brewer, alcohol)

Attributes of R = attributes of E
Key of R = Key of E

Rule2: Relationship → Relation



⇓

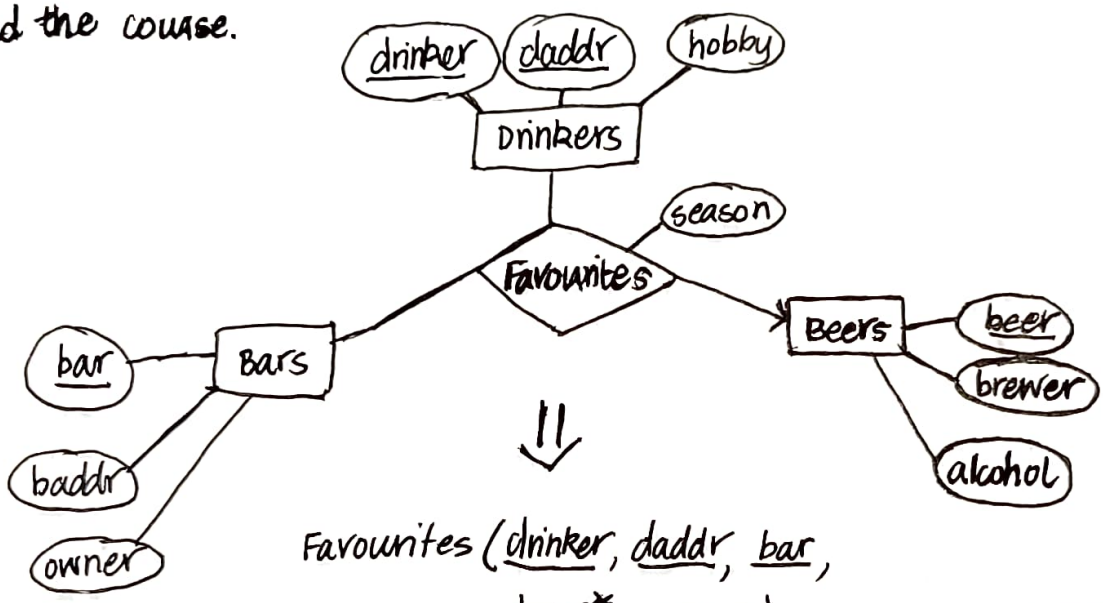
Enrolls(<u>sid</u>, sname, major, birthday, <u>number</u>, title, credit, term, grade)

⇓

Enrolls(<u>sid</u>, <u>number</u>, term, grade)

<u>sid</u> is enough to then find student and other information

<u>number</u> is enough to then find the course.



⇓

Favourites(<u>drinker</u>, <u>daddr</u>, <u>bar</u>, <u>beer</u>, season)

beer not needed as key b/c there is at most 1 beer per drinker and bar.

⇓

Favourites(<u>drinker</u>, <u>daddr</u>, <u>bar</u>, ~~beer~~ beer, season)

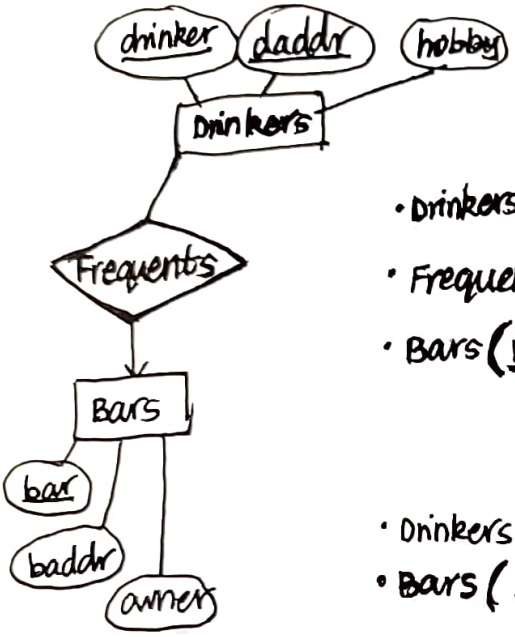
Rule 2 : Relationship → Relation

Rule: Translating relationship X of $E_1, \ldots, E_n$ to relation R

- Attributes of R = key attributes of $E_1, \ldots, E_n$ plus attributes of X.
- Key of R = Key of $E_1, \ldots, E_n$ except those "arrowed" entities.

Rule 3 : Combining Many-One Relationships

Rule: Combine the relation of a many-one relationship with the relation of the "many"-side entity set.



- Drinkers(drinker, daddr, hobby)
- Frequents (drinker, ~~bar~~, daddr, bar)
- Bars (name, baddr, owner)

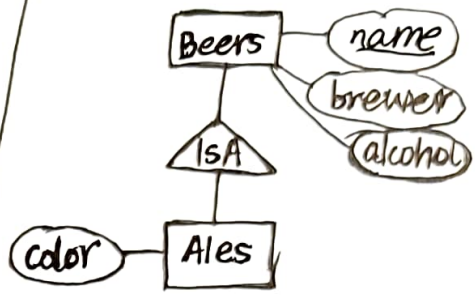
⇓

- Drinkers (drinker, daddr, hobby, bar)
- Bars (name, baddr, owner)

2.5 Translating Subclasses

Expressing subclasses in Relations

- Relational model is not object-oriented.
- Object features were motivation for new models.
  - object-oriented.
  - object-relational.



~~How to~~

- We need a relation for Beers. This is normal.

  Beers (name, brewer, alcohol)

Several Options -- None Perfect

- ER Approach
- Object-Oriented Approach
- Null-Value Approach

- consider them by criteria:
  - performance for common queries
  - space need for typical data.

Option 1: The 'ER Approach'

storing subclass as a relation, with only additional attributes. Link to superclass via key.

Ex. Beers(name, brewer, alcohol), Ales(name, color)

name	brewer	alcohol
Sam A	BostonB	4.9
Goose IPA	Goose	5.0
SummerAle	BostonB	6.0

name	color
Goose IPA	Golden
SummerBle	Dark

Option 2: The "OO Approach"

- Inheritance: Subclass inherits Attributes from superclass.

Ex:
- Beers (name, brewer, alcohol)
- Ales (name, brewer, alcohol, color)

Ales are only stored in Ales table and not Beers tables.

Option 3: "Null-Value Approach"

- One Relation for superclass and subclass.

Ex: Beers (name, brewer, alcohol, aleColor)

name	brewer	alcohol	alecolour
Sam A.	Boston Beer	4.0	Null
Goose IPA	Goose Island	5.0	Golden
Summer Ale	Boston Beer	6.0	Dark

What would work faster?

- Find the colours of ales made by some company, say, Boston Beer.

	#Tables	Size
ER Approach	2	Small
OO Approach	1	Small ☆
Null-Value Approach	1	Large

- Depends on query

Find all beers made by Boston beer

ER Approach ✗

2.6 Post-Relational - Object Base Modelling

Impedence Mismatch:
Database vs Application Programming

Object (of class) vs Tuple (of relation):

- Object Identifiers: Object referenced by identifiers (pointers) while tuples by values.
- Nesting: Object can be nested to contain objects while tuple contains only simple values as attributes.
- Methods: Object has methods while tuple contains only attributes.
- Inheritance: Classes can form subclass hierarchy while relations cannot.
- Encapsulation: object can hide internal representation.

Meeting Programming Paradigms

- Object-Oriented Data Model
  - Extending OO programming languages with database functions.
  - Current Systems:
    - Objectivity/DB
    - DB4objects
    - Intersystems CACHE
- Object-Relational Data Model
  - Extending RDBMS with object features
  - Current systems: PostgreSQL
    - most RDBMS supports object features to various extents.

Object-Relational: PostgreSQL

Table Inheritance:

```
CREATE TABLE Beers(
  name text,
  brewer text,
  alcohol float)

CREATE TABLE Ales(
  color char(10)) INHERITS(Beers);
```

2.7 - Post-relational - NoSQL Modelling

Dealing with data in various new settings
→ No SQL

- New kinds of data: Web data, social networks, scientific data.
- New requirements:
  - Volume → Scalability
    - Handling extremely large data.
    - Handling extremely ~~large~~ many users.
  - Variety → One model may not fit all
    - Handling very simple to complex data.
- NoSQL databases:
  - Originally "non SQL" or "not relational"
  - Now "not only SQL"

NoSQL Data Models

- Key-Value Model
  - Redis, Berkeley DB
- Document Model
  - MongoDB, CouchDB
  - JSON is a popular document model
- Graph Model
  - Neo4j, OrientDB.

Key-Value Model

- DB = key-value pairs
- Key: Any binary sequence given/named by programmer.
- Value: string, or more complex types list, hash, set
- Data model is effectively managed at applications.
- Good for simple data with simple look ups.

Graph Model

- Database = Graph
- Node = Entities. Edges = Relationships
- Relationship as first class citizens.

Document Model

- DB = Collections of Documents
- Document encoded in nested structure, e.g. XML, JSON.
- Documents do not necessarily share the same schema.
  - But you can enforce a schema (or document validation rules)

mongoDB, CouchDB

JSON: JavaScript Object Notation

- Standard for serializing data object in human-readable format.
- Popularized as common format for browser server data exchange.
- Then backend databases start to store data as JSON too.
  - And the idea of document databases emerged.
- Used by different languages/systems beyond JavaScript.

JSON Documents

- Human Readable
- Value: basic types are strings, number, booleans, null.
- Object: {field-value pair} i.e. set of field-value pairs.
- Array: [value]
- Nesting: Value can be an embed objects or referenced objects

JSON Schema

- We can mix different kinds of documents in a collection.
- We can also specify the structure of JSON data by a JSON schema.
- In JSON format. Human readable.
- Define structures
  - set of properties (fields)
  - Type for each property
  - Constraint for each property